



How Do Maps, Hash Maps and Sets Work?

In this lesson, we'll go over maps, hash maps, and sets. But before we do that, let's define Abstract Data Types.

An Abstract Data Type (ADT) is a conceptual representation of a data type, including what operations can be performed on the data and the properties of that data.

Abstract Data Types are like blueprints that describe **what** operations can be performed, not **how** they are performed. They separate the interface from the actual implementation of the operations.

A **map** is an ADT that manages collections of key-value pairs and their operations in a very specific and efficient way.

In a map, every value is associated with a specific key.

One of the key characteristics of maps is that every key must be unique. This uniqueness allows for direct lookups, which makes the process of retrieving information much more efficient.

Only keys must be unique, values can be repeated.

The map Abstract Data Type also defines important operations, such as inserting key-value pairs, getting the value associated with a key, updating the value associated with a key, removing a key-value pair, and checking if a key exists in the map.

It doesn't actually specify how these operations should be performed, it just lists them as part of the available operations of the data type.

A **hash map**, also known as a hash table, is a concrete implementation of the map Abstract Data Type.

Hash maps use a technique called "hashing" to perform common operations very efficiently.

Hashing essentially works by generating a hash value for each element using a hash function.

The hash value is generated based on the key of the key-value pair and it's used to calculate an index in an underlying array, the actual data structure where the key-value pairs are stored.

But you might be asking yourself: What happens if two keys result in the same index?

Hash maps solve these collisions with clever strategies.

One option is to use the "chaining" strategy, where each array index points to a linked list (another data structure), where all the elements with the same index are stored.

Another strategy is to use "open addressing", which involves searching for the next available index in the array based on a predefined search sequence.

The average case time complexity of hash maps is "Constant Time" $O(1)$ for inserting, retrieving, and deleting key-value pairs.

The worst case time complexity of these operations is Linear Time $O(n)$, which occurs when there are many hash collisions, so the collision resolution strategy has to be applied multiple times.

The space complexity of inserting into a hash map is constant $O(1)$ on the average case, a constant amount of memory to store the new pair. However, in the worst case, it can have linear space complexity $O(n)$ due to a resizing operation of the underlying array. In general, removing an element has a constant space complexity $O(1)$.



Python's **dictionaries** are implemented as hash maps behind the scenes.

To create a Python dictionary, you just need to write the key-value pairs within curly brackets and separate them with a comma. Each key should be separated from its corresponding value with a colon.

```
my_dictionary = {  
    'A': 1,  
    'B': 2,  
    'C': 3  
}
```

In this code, `'A'` is the key and `1` is the value:

```
'A': 1
```

Alternatively, you can use `dict()`:

```
my_dictionary = dict(A=1, B=2, C=3)
```

You can get the value through its corresponding key:

```
my_dictionary['A'] # 1
```

You can also update the value associated with a key:

```
my_dictionary['A'] = 4
```

And you can remove a key-value pair:

```
del my_dictionary['A']
```

You can also check if a key is in the dictionary (or not):

```
'C' in my_dictionary
```

And you can call these methods to get the keys, values, and items of the dictionary, respectively.

```
my_dictionary.keys()  
my_dictionary.values()  
my_dictionary.items()
```

Great. Now that you know more about maps and hash maps, let's talk about sets.

Sets are unordered collections of unique elements.

Let's break this concept down into its key components:

- Sets are unordered. The elements of a set are not stored in any specific order, so you cannot access them through indices.



One of the main advantages of sets is that they guarantee that the elements will be unique (no duplicates). This is why they are often used to remove duplicates from lists and other data structures.

They are also dynamic. They can adjust to the number of elements that are currently stored. This makes them quite powerful.

The average case time complexity of adding, removing, getting the length of the set, and checking if an element is in the set is "Constant Time" $O(1)$, which is very efficient.

Since sets are implemented as hash tables, the worst case time complexity of adding, removing, and checking membership is "Linear Time" $O(n)$. This may occur when there are multiple hash collisions, transforming the hash table into something similar to a linear data structure, where n scans are required to find the key.

In terms of space complexity, in the average case, inserting an element would have constant complexity $O(1)$, with a new unique element requiring a constant amount of memory. However, in the worst case, there could be a resizing operation of the underlying array, which could take linear space complexity $O(n)$. In general, removing an element would take constant space complexity $O(1)$.

Python has a built-in `set` data structure that you use to work with sets in your programs.

Behind the scenes, Python sets are implemented using a hash table where only the keys are stored, without any associated values.

Sets can only store objects of immutable data types because their hash values always remain the same. In contrast, the hash values of mutable objects can change when they are mutated. That's why they cannot be part of sets. If the hash value of an object stored in the set changes, the program would not be able to find it anymore.

To define a set in Python, you just need to surround the elements with curly brackets and separate them with commas:

```
numbers = {1, 2, 3, 4}
```

To create an empty set, you can call `set()`:

```
numbers = set()
```

Note that if you use empty curly brackets, this will automatically create a Python dictionary, not a set, so you must call the `set()` function to create an empty set.

You can add an element to a set with the `.add()` method:

```
numbers.add(5)
```

You can also remove elements from the set with the `.remove()` method:

```
numbers.remove(5)
```

This will throw a `KeyError` if the element is not found. But if you don't want to throw an error in that case, you can use the `.discard()` method instead.

The `.pop()` method returns an arbitrary element from the set, while the `.clear()` method removes all elements from the set.

You can test if an element is in a set with the `in` operator:



Python also support set operations, including union, difference, symmetric difference, and intersection, which you can perform with these methods:

```
set_a = {1, 2, 3, 4}
set_b = {2, 3, 4, 5, 6}

set_a.union(set_b)
set_a.intersection(set_b)
set_a.symmetric_difference(set_b)
set_a.difference(set_b)
```

Or with their equivalent operators:

```
set_a | set_b
set_a & set_b
set_a ^ set_b
set_a - set_b
```

The average case time complexity for adding, removing, and testing membership is "Constant Time" $O(1)$.

The worst case time complexity for these operations is "Linear Time" $O(n)$ because of the hash map's worst case collision scenario.

You can also check if a set is a subset or superset of another one:

```
set_a.issubset(set_b)
set_a.issuperset(set_b)
```

In general, you should use sets when you need to store a collection of unique items and frequently check for the presence of an item.

Maps, hash maps, and sets are powerful data structures designed for efficient data organization and retrieval. Each one of them has its own unique characteristics and use cases. As a developer, you will need to choose the best one for your project.

Questions

What is the fundamental difference in the type of data stored by a hash map (or map) compared to a set?

Hash maps store ordered collections, while Sets store unordered collections.

Hash maps store unique key-value pairs, while Sets store unique individual elements.

Correct!

Hash maps cannot store duplicate values, while Sets can.

Sets are used for numerical data, while hash maps are for textual data.



Python Certification

Working with Common Data Structures

They keep all elements sorted, enabling fast binary search.

They store elements in a linked list, allowing quick traversal.

They use a hash function to compute a direct memory location for elements.

Correct!

They always store a very small number of elements, making all operations fast.

In the context of hash maps and sets, what is a "hash collision"?

When an element is successfully found after a search.

When an element is inserted at the very beginning of the collection.

When the hash map runs out of memory and needs to resize.

When two different keys or elements produce the same hash value.

Correct!

Check your answer

Ask for Help